



수학 시뮬레이터 제작 보고서

수행평가

《시뮬레이션 파일 저장하는 방법》

- ① 최종 시뮬레이터의 html 코드를 전체 복사한다.
- ② 윈도우 메모장을 켜 후 위의 코드를 붙여넣기 한다.
- ③ 메모장에서 파일 - 다른 이름으로 저장 - 각각 본인이름1.html, 본인이름2.html으로 저장
(예 : 이현진1.html, 이현진2.html으로 꼭 확장자 붙여서 저장)

[첫 번째 시뮬레이터 제작 보고서]

1. 내가 선택한 첫 번째 문제 (캡처해서 붙여넣기 해도 됨)

KAIST 2020 학교장 추천전형 구술면접 1번 문항

자연수 n 에 대해 0보다 크거나 같은 값을 갖는 확률변수 X_n 의 확률밀도함수가

$$f_n(x) = c_n x^n e^{-x}, x \geq 0$$

으로 주어졌다. (e 는 자연로그의 밑, c_n 은 n 에 따라 달라지는 양의 상수이다.)

단, 모든 n 에 대해서 $\lim_{x \rightarrow \infty} x^n e^{-x} = 0$ 임이 알려져 있다. (총 5점)

(1) $n=2$ 인 경우인 $f_2(x)$ 가 확률밀도함수가 되는 상수 c_2 의 값을 구하시오. (2점)

(2) 자연수 n 과 고정된 양의 상수 $r > 0$ 에 대해서 확률 $P(a \leq X_n \leq a+r)$ 가 최대가 되는 양수 a 를 r 과 n 을 사용하여 표현하고 최대인 근거를 설명하시오. (3점)

2. 이 문제의 풀이 (반드시 손글씨로 작성하여 캡처하거나 패드에 손글씨로 작성한 것을 붙여넣으세요.)

$$(1) \int_{-\infty}^{\infty} f_2(x) dx = 1.$$

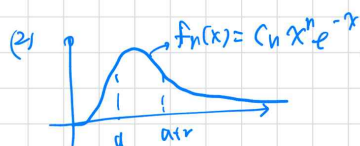
$$\therefore \int_{-\infty}^{\infty} c_2 x^2 e^{-x} dx$$

$$= c_2 (-x^2 - 2x - 2) e^{-x} \Big|_{-\infty}^{\infty}$$

$$= 2c_2 = 1.$$

$$\therefore c_2 = 1/2$$

u	v'
e^{-x}	x^2
$-e^{-x}$	$2x$
e^{-x}	2
$-e^{-x}$	



그러므로 $f_n(a) = f_n(a+r)$ 에 대해 양쪽을 곱해주고,

$f_n(a)$ 는 $c_n a^n e^{-a}$, $f_n(a+r)$ 은 $c_n (a+r)^n e^{-(a+r)}$ 이므로 $P(a \leq X_n \leq a+r)$ 은 최대이다.

$$\therefore c_n a^n e^{-a} = c_n (a+r)^n e^{-(a+r)}$$

$$\therefore \left(\frac{a+r}{a}\right)^n = e^r \therefore 1 + \frac{r}{a} = e^{r/n} \therefore a = \frac{r}{e^{r/n} - 1}$$

3. 이 문제가 시뮬레이션이 필요하다고 느낀 이유를 서술하세요.

이 문제의 메인 아이디어는 양쪽 끝의 확률밀도 함수 값이 같을 때에 최대가 된다는 점이다. 이는 적분의 증감을 통해 설명되는 부분이다. 순수하게 적분을하고자 해도 답은 도출되나, 아이디어를 통해서 해결한다면 손쉽게해결할 수 있다. 그렇기에 이와 비슷한 아이디어를 사용하는 문제가 등장하였을 때 가장 먼저 함숫값을 고려해보는 아이디어를 떠올리게 하기 위해서 시뮬레이션을 통해 이를 정리하여 오래 기억하고자 하였으며, 아이디어가 잘 이해되지 않는 사람들에게는 이 시뮬레이션이 이해를 돕는데 큰 도움이 될 것이라 생각된다.

4. 사용한 AI 플랫폼을 모두 적으세요.

Gemini 3.1

5. 시뮬레이터를 만들기 위한 최초의 프롬프트를 작성하세요.

이 문제에서 이해를 쉽게 할 수 있도록 돕는 시뮬레이션을 html로 만들거야. 함수식에 해당하는 그래프를 그리고, 왼쪽 끝 값을 사용자가 조절할 수 있게 하다가 양쪽 끝 두 함숫값의 크기가 같을 때 값이 최대가 됨을 보여줄 수 있도록 확률을 보여주다가 최댓값이 되면은 초록색으로 변하도록 해줘

6. 최초 시뮬레이터의 모습을 캡처해서 붙여넣으세요.

```
import React, { useState, useEffect, useRef } from 'react'; import { Info, Play, MoveHorizontal } from 'lucide-react'; // 팩토리얼 계산 함수 const factorial = (num) => { if (num <= 1) return 1; return num * factorial(num - 1); }; // 확률밀도함수 f_n(x) const fn = (x, n) => { const cn = 1 / factorial(n); return cn * Math.pow(x, n) * Math.exp(-x); }; // 주어진 n, r에 대해 확률이 최대가 되는 최적의 a 값을 이분법으로 찾음 const findOptimalA = (n, r) => { let low = 0.001; let high = 20.0; for (let i = 0; i < 60; i++) { let mid = (low + high) / 2; let f1 = fn(mid, n); let f2 = fn(r, n); if (Math.abs(f1 - f2) < 0.000001) return mid; else if (f1 > f2) high = mid; else low = mid; } return (low + high) / 2; }; // 사다리꼴 공식을 이용한 수치 적분 (확률 계산) const calculateProbability = (n, a, r) => { let area = 0; const steps = 200; const dx = r / steps; for (let i = 0; i < steps; i++) { const x1 = a + i * dx; const x2 = a + (i + 1) * dx; area += ((fn(x1, n) + fn(x2, n)) * dx) / 2; } return area; }; export default function App() { const [n, setN] = useState(2); const [r, setR] = useState(3.0); const [a, setA] = useState(0.5); const canvasRef = useRef(null); // 최적의 a값과 현재 a값의 오차를 바탕으로 최대 상태 판별 const optimalA = findOptimalA(n, r); const isMax = Math.abs(a - optimalA) < 0.05; // 오차 허용 범위 const currentProb = calculateProbability(n, a, r); const fa = fn(a, n); const far = fn(a + r, n); // 그래프 그리기 useEffect(() => { const canvas = canvasRef.current; if (!canvas) return; const ctx = canvas.getContext('2d'); // 캔버스 레티나 디스플레이 대응 및 크기 설정 const dpr = window.devicePixelRatio || 1; const rect = canvas.getBoundingClientRect(); canvas.width = rect.width * dpr; canvas.height = rect.height * dpr; ctx.scale(dpr, dpr); const width = rect.width; const height = rect.height; // 그리기 영역 설정 const padding = { top: 30, right: 30, bottom: 40, left: 50 }; const xMax = 12; // x축 최대 표시값 const yMax = 0.45; // y축 최대 표시값 (n=1~4 범위 내에서 충분한 값) const toX = (val) => padding.left + (val / xMax) * (width - padding.left - padding.right); const toY = (val) => height - padding.bottom - (val / yMax) * (height - padding.top - padding.bottom); ctx.clearRect(0, 0, width, height); // 1. 축 및 그리드 그리기 ctx.strokeStyle = '#e5e7eb'; // 옅은 회색 ctx.lineWidth = 1; ctx.beginPath(); for (let i = 0; i <= xMax; i++) { ctx.moveTo(toX(i), padding.top); ctx.lineTo(toX(i), height - padding.bottom); } for (let j = 0; j <= yMax; j++) { ctx.moveTo(padding.left, toY(j)); ctx.lineTo(width - padding.right, toY(j)); } ctx.stroke(); // X, Y 기본 축 (진하게) ctx.strokeStyle = '#9ca3af'; ctx.lineWidth = 2; ctx.beginPath(); ctx.moveTo(padding.left, height - padding.bottom); ctx.lineTo(width - padding.right + 10, height - padding.bottom); // X축 ctx.moveTo(padding.left, height - padding.bottom); ctx.lineTo(padding.left, padding.top - 10); // Y축 ctx.stroke(); // 2. 확률 영역 (적분 면적) 칠하기 ctx.beginPath(); ctx.moveTo(toX(a), toY(0)); for (let x = a; x <= a + r; x += 0.05) { ctx.lineTo(toX(x), toY(fn(x, n))); } ctx.lineTo(toX(a + r), toY(0)); ctx.closePath(); ctx.fillStyle = isMax ? 'rgb(34, 197, 94, 0.35)' : 'rgb(59, 130, 246, 0.2)'; // 최대면 초록, 아니면 파랑 ctx.fill(); // 3. 확률밀도함수 곡선 그리기 ctx.beginPath(); for (let x = 0; x <= xMax; x += 0.1) { const px = toX(x); const py = toY(fn(x, n)); if (x === 0) ctx.moveTo(px, py); else ctx.lineTo(px, py); } ctx.strokeStyle = '#f2f2f2'; ctx.lineWidth = 2.5; ctx.stroke(); // 4. a와 a+r 위치의 수직선 및 높이 비교선 const drawDashedLine = (x1, y1, x2, y2, color) => { ctx.beginPath(); ctx.moveTo(x1, y1); ctx.lineTo(x2, y2); ctx.strokeStyle = color; ctx.lineWidth = 1.5; ctx.setLineDash([5, 5]); ctx.stroke(); ctx.setLineDash([]); } const pAX = toX(a); const pAY = toY(fa); const pAR_X = toX(a + r); const pAR_Y = toY(far); // 수직선 drawDashedLine(pAX, toY(0), pAX, pAY, '#9ca3af'); drawDashedLine(pAR_X, toY(0), pAR_X, pAR_Y, '#9ca3af'); // f(a)와 f(a+r) 높이 비교 가로선 (적분 포인트) const lineColor = isMax ? '#16a34a' : '#f44444'; // 값이 같으면 초록, 아니면 빨강 drawDashedLine(pAX, pAY, pAR_X, pAR_Y, lineColor); // 5. 양 끝 점 찍기 const drawPoint = (x, y, color) => { ctx.beginPath(); ctx.arc(x, y, 5, 0, Math.PI * 2); ctx.fillStyle = color; ctx.fill(); ctx.strokeStyle = 'black'; ctx.lineWidth = 2; ctx.stroke(); } drawPoint(pAX, pAY, lineColor); drawPoint(pAR_X, pAR_Y, lineColor); // 6. 텍스트 라벨 (x축, 함수식) ctx.fillStyle = '#4b5563'; ctx.font = '13px sans-serif'; ctx.textAlign = 'center'; ctx.fillText('a', pAX, height - padding.bottom + 10); ctx.fillText('a+r', pAR_X, height - padding.bottom + 10); // y축 라벨 ctx.textAlign = 'right'; ctx.fillText('f(x)', padding.left - 10, padding.top); ctx.fillText('x', width - padding.right + 15, height - padding.bottom + 4); }, [n, a, r, isMax]); return ( />
```

확률밀도함수 최대 확률 구간 시뮬레이터

목표: 아래 슬라이더를 움직여 시작 위치 \$a\$를 변경해보세요. 구간 \$[a, a+r]\$의 넓이(확률)가 최대가 되려면 양 끝점의 높이가 \$f_n(a)\$와 \$f_n(a+r)\$이 같아져야 함을 확인할 수 있습니다.

```
/* 메인 그래프 영역 */
/* Max 표시 범위 */ isMax && {
  ✨ 확률 최대 조건 달성! f(a) = f(a+r) ✨

  ))
  /* 시각적 결과 피드백 요약 */
  현재 구간 확률 $P(a \le X_n \le a+r)$
  (currentProb * 100).toFixed(2)%
  /* 컨트롤 및 정보 패널 */
  /* 파라미터 조절기 */
```

변수 조절하기

```
/* a 슬라이더 (사용자 직접 조절) */
시작 위치 $a$ {a:toFixed(2)}
setA(parseFloat(e.target.value)) className="w-full h-2 bg-gray-200 rounded-lg appearance-none cursor-pointer accent-blue-600" />

/* r 슬라이더 */
구간 길이 $r$ {r:toFixed(1)}
setR(parseFloat(e.target.value)) className="w-full h-2 bg-gray-200 rounded-lg appearance-none cursor-pointer accent-gray-500" />
```

7. 최초 시뮬레이터에서 수정해야 할 사항을 서술하세요.

①텍스트 깨짐

수식의 텍스트와 엔터 간격 등이 모두 깨져서 등장하였다.

②그래프 비 등장

html에 요구한 그래프가 깨져 등장하지 않았다.

③Gemini 내부 Canvas에서만 잘 돌아가는 시뮬레이터

이것이 가장 근본적인 문제이다. import 하는 라이브러리 등이 html 만 돌렸을 때 돌아가지 않는다는 문제점이 존재하였다.

8. 수정하기 위한 프롬프트를 (2번째 ~ n번째) 모두 작성하세요.

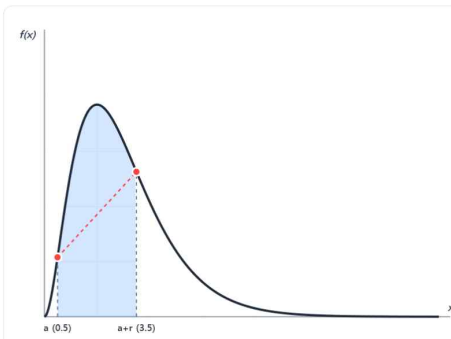
②뭔가 이게 html로 코드를 옮겨서 실행하니까 텍스트 박스나 그래프들이 깨져버렸어. 수정해줘. 미리보기에선 잘 나오지만

③n도 드래그해서 수정할 수 있도록 하고, 지금 html에서 보이는 사이즈가 너무 작아. 화면 안에 들어오는 사이즈를 키워줘

9. 최종 시뮬레이터의 모습을 캡처해서 붙여넣으세요.

▷ 확률밀도함수 최대 확률 구간 시뮬레이터

① 목표: 아래 시뮬레이터를 움직여 시작 위치 a 와 구간 길이 r 의 높이(확률)가 최대가 되려면 양 끝점의 높이 $f_n(a)$ 와 $f_n(a+r)$ 이 같아져야 함을 직관적으로 확인할 수 있습니다.



현재 구간의 확률 $\int_a^{a+r} f(x) dx$

66.48%

↔ 변수 조절 패널

함수 차수 n 2

그래프의 모양이 크게 바뀝니다

시작 위치 a 0.50

구간 길이 r 3.0

실시간 함수값 비교

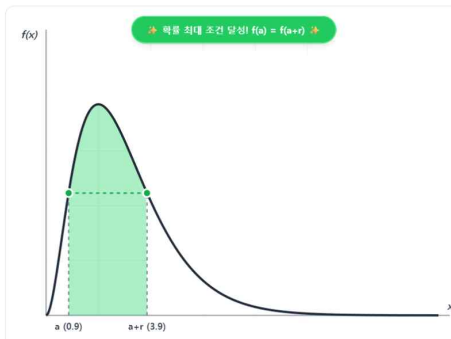
좌측 끝 $f_n(a)$ 0.07582

우측 끝 $f_n(a+r)$ 0.18496

오른쪽 점이 더 높습니다
높이 차이: 0.10914

▷ 확률밀도함수 최대 확률 구간 시뮬레이터

① 목표: 아래 시뮬레이터를 움직여 시작 위치 a 와 구간 길이 r 의 높이(확률)가 최대가 되려면 양 끝점의 높이 $f_n(a)$ 와 $f_n(a+r)$ 이 같아져야 함을 직관적으로 확인할 수 있습니다.



현재 구간의 확률 $\int_a^{a+r} f(x) dx$

68.42%

↔ 변수 조절 패널

함수 차수 n 2

그래프의 모양이 크게 바뀝니다

시작 위치 a 0.86

구간 길이 r 3.0

실시간 함수값 비교

좌측 끝 $f_n(a)$ 0.15683

우측 끝 $f_n(a+r)$ 0.15683

두 함수값이 일치합니다
현재 구간에서 최대 확률을 가집니다

10. 자신이 만든 시뮬레이터를 처음 본 사람이 사용할 수 있도록 사용 방법을 서술하세요.

1. 수동 조작 및 변화 관찰 (변수 슬라이더 제어)

-조작 방법: 우측 패널의 슬라이더를 마우스로 드래그하여 함수 차수 n , 시작 위치 a , 구간 길이 r 을 수동으로 제어함.

-기능: n 값에 따른 확률밀도함수 그래프 개형의 변화를 확인할 수 있으며, a 와 r 을 조작함에 따라 구간(푸른색 영역)의 넓이와 양 끝점의 높이가 실시간으로 변하는 과정을 직관적으로 관찰할 수 있음.

2. 최적 위치 자동 탐색 (자동 맞추기 기능)

-조작 방법: 패널 하단의 '정확한 최댓값 위치 자동 맞추기' 버튼 클릭.

-기능: 현재 설정된 함수 차수 n 과 구간 길이 r 을 기준으로, 구간의 넓이(확률)가 최대가 되는 최적의 시작 위치 a 를 시뮬레이터가 내부적으로 계산하여 즉시 이동시킴. 이를 통해 최댓값이 형성되는 순간의 그래프 형태를 시각적으로 빠르게 그리고 자동적으로 확인할 수 있음.

3. 최대 확률 조건 분석: 양 끝점의 함수값 일치 탐색

-조작 방법: a 슬라이더를 좌우로 조정하며 우측 '실시간 함수값 비교' 상태 창 확인.

-기능: 좌측 끝점 $f_n(a)$ 와 우측 끝점 $f_n(a+r)$ 의 높이 차이를 상태 메시지(좌/우측 점이 더 높습니다)로 알림 발생. 두 함수값이 일치하는 위치에 도달할 시 화면 내 상단에 녹색 알림 배지가 발생하고 확률 텍스트가 전환됨. 이를 통해 면적이 최대가 되는 조건이 $f_n(a) - f_n(a+r) = 0$ 임을 직접 검증할 수 있음.

11. 시뮬레이터가 자신이 의도한대로 문제를 파악하거나 풀이과정에 대한 힌트를 얻거나 풀이에 대한 검토를 하는 등에 도움이 되었는지, 도움이 되거나 되지 않는 부분에 대해서 구체적으로 서술하세요.

이 시뮬레이터는 풀이를 검토하는데에 도움이 되었다. 확률 밀도함수가 증가 후 감소하는 구형이게 때문에 구간이 일정할 때에는 양 쪽 함수값이 일치할 때에 최대가 된다는 인사이트를 잘 보여준다,

[도움이 된 부분]

문항이 의도하는 부분이 내가 생각한 것과 맞는 지 확인할 수 있다.

[도움이 되지 않는 부분]

너무 답을 알려준다는 느낌이 너무 강하게 나기에 힌트보다는 치트키를 사용하는 것과 유사하다는 것이 단점이다. 문제를 풀 때에 약간의 힌트를 거쳐 답을 구하는 것이 바로 답지를 보는 것 보다 수학 실력 증진에 있어 도움되다는 점을 경험적으로 알고 있기에 이 지점을 고려하지 못 한 점이 한계이자 아쉬운 부분이다.

[대안]

이를 극복하기 위해서는 양 쪽 끝 점의 구속을 분리하고 사용자가 직접 양쪽 끝 값의 변화에 따른 확률의 값을 관찰하다가 r 에 대해 일정한 차이를 가질 수 있도록 바꿈으로서 보다 단계적인 이해를 하도록 할 수 있을 듯 하다.

[두 번째 시뮬레이터 제작 보고서]

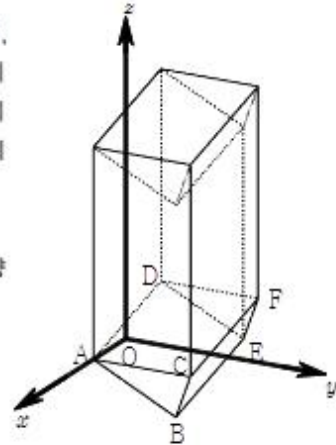
1. 내가 선택한 두 번째 문제 (캡처해서 붙여넣기 해도 됨)

연세대학교 2020 논술고사 4번

2. 문항 및 제시문

[제시문]

그림과 같이 좌표공간에서 $A(1, 0, 0)$, $B(0, 1, -2)$, $C(1, 2, 0)$, $D(-1, 0, 1)$, $E(-2, 1, -1)$, $F(-1, 2, 1)$ 를 꼭짓점으로 하는 삼각기둥 $ABC-DEF$ 를 z 축의 방향으로 6만큼 평행이동하는 동안 삼각기둥 $ABC-DEF$ 가 그리는 다면체를 V 라 하자. 다음 물음에 답하시오.

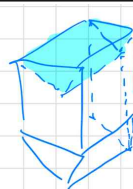


[문제 4-1] 다면체 V 와 평면 $z=3$ 이 만나서 생기는 단면의 모양과 넓이를 구하시오.

[문제 4-2] 다면체 V 의 부피를 구하시오.

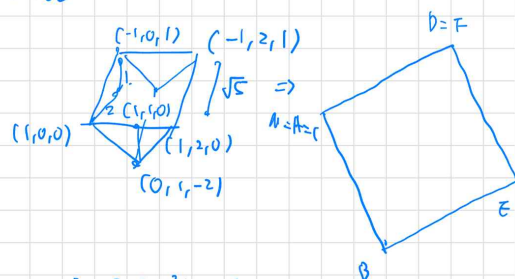
2. 이 문제의 풀이 (반드시 손글씨로 작성하여 캡처하거나 패드에 손글씨로 작성한 것을 붙여넣으세요.)

[4-2]

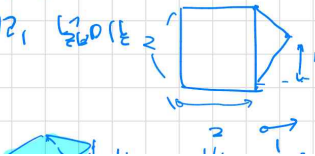


본여리는 z 방향의 z 축을 따라
다면체로 나뉘다

다른 색 부분은 z 축 삼각기둥의 부피이며, 나머지는 그 높이가
모두 6으로 같기때 카탈라리에 리해 z 이 이 비례 $6 \times 5 = 30$.
삼각기둥은



[4-1] $ABC-DEF$ 에 대해 xy 평면상의 점과
2차원은 대칭성 모양과 같다. $\therefore A, C, F, E, D$ 가
본여리는 정이, z 축을 z
 $\therefore 5$ 이다



$\therefore$ 이 각 삼각기둥이다.
 $\therefore 2 \cdot \sqrt{5} \cdot \frac{1}{2} \cdot \sqrt{5} = 5. \therefore 5$.
 $\therefore$ 총합은 35이다.

3. 이 문제가 시뮬레이션이 필요하다고 느낀 이유를 서술하세요.

이 문제를 선정한 본인은 이 문제를 처음 보고 틀렸다. 그 이유는 기울어진 삼각형 부분이 험쓸고 지나가는 부분의 모양을 상상하기 까다롭기 때문이다. 이런 문제는 이 문제에 예상 소요시간이 30분으로 잡혀있는 데에도 잘 반영되어있다. 이런 어려움이 나만 느끼는 것이 아니라 생각되기에 험쓸고 지나가는 부피를 구할 수 있는 인사이트를 제공하는 시뮬레이션을 만들고자 하였다.

4. 사용한 AI 플랫폼을 모두 적으세요.

Chat GPT 5.5 thinking model

5. 시뮬레이터를 만들기 위한 최초의 프롬프트를 작성하세요.

이 문제를 직관적으로 해결할 수 있는 시뮬레이터를 html로 만들자. 시뮬레이터에는 주어진 도형이 평행이동하는 h에 따라 발생하는 자취를 보여주고, 그 아래에는 3차원으로 된 이 도형의 모습을 그려주고, 이를 구하기 위해 나눠지는 도형들의 부분들을 나눈 3D 한 개, 그리고 각 도형들을 각각 3D플롯을 진행하고, 그들의 부피를 계산하는 과정을 보여줘.

6. 최초 시뮬레이터의 모습을 캡처해서 붙여넣으세요.

삼각기둥 ABC-DEF의 z축 평행이동 자취 시뮬레이터

초기 삼각기둥 $P = ABC - DEF$ 를 z 축 방향으로 h 만큼, $0 \leq h \leq 6$, 평행이동할 때 생기는 자취 V 와 평면 $z = 3$ 의 단면을 시각화합니다.

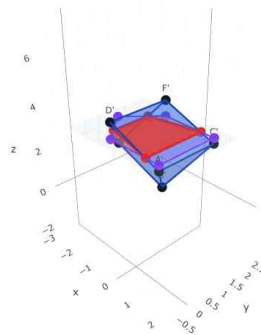
1. **비판:** 열거 원리의 오용기법 2. **비판:** 열거 $n = 3$ 절단법 3. **비판:** 지금까지 줄고 지나간 단면 자취 4. **비판:** 열거 단면 오작동

1. $\hbar$ 에 따른 자취 관찰

$h = 2.74$ 일 때의 현재 모형과 $z=3$ 자취

● 重要人物

氏名	役職
山本 武彦	代表取締役社長
佐々木 孝一	取締役専任執行役員
田中 一郎	取締役専任執行役員
鈴木 健二	取締役専任執行役員
高橋 三郎	取締役専任執行役員
伊藤 四郎	取締役専任執行役員
渡辺 五郎	取締役専任執行役員
松本 六郎	取締役専任執行役員
石川 七郎	取締役専任執行役員
山口 八郎	取締役専任執行役員
北村 九郎	取締役専任執行役員
上野 十郎	取締役専任執行役員
小島 十一郎	取締役専任執行役員
大塚 十二郎	取締役専任執行役員
森田 十三郎	取締役専任執行役員
山崎 十四郎	取締役専任執行役員
斎藤 十五郎	取締役専任執行役員
高木 十六郎	取締役専任執行役員
岡田 十七郎	取締役専任執行役員
宮田 十八郎	取締役専任執行役員
林 十九郎	取締役専任執行役員
森 二十郎	取締役専任執行役員
山本 二十一郎	取締役専任執行役員
佐々木 二十二郎	取締役専任執行役員
田中 二十三郎	取締役専任執行役員
鈴木 二十四郎	取締役専任執行役員
高橋 二十五郎	取締役専任執行役員
伊藤 二十六郎	取締役専任執行役員
渡辺 二十七郎	取締役専任執行役員
松本 二十八郎	取締役専任執行役員
石川 二十九郎	取締役専任執行役員
山口 三十郎	取締役専任執行役員
北村 三十一郎	取締役専任執行役員
上野 三十二郎	取締役専任執行役員
小島 三十三郎	取締役専任執行役員
大塚 三十四郎	取締役専任執行役員
森田 三十五郎	取締役専任執行役員
山崎 三十六郎	取締役専任執行役員
斎藤 三十七郎	取締役専任執行役員
高木 三十八郎	取締役専任執行役員
岡田 三十九郎	取締役専任執行役員
宮田 四十郎	取締役専任執行役員
林 四十一郎	取締役専任執行役員
森 四十二郎	取締役専任執行役員
山本 四十三郎	取締役専任執行役員
佐々木 四十四郎	取締役専任執行役員
田中 四十五郎	取締役専任執行役員
鈴木 四十六郎	取締役専任執行役員
高橋 四十七郎	取締役専任執行役員
伊藤 四十八郎	取締役専任執行役員
渡辺 四十九郎	取締役専任執行役員
松本 五十郎	取締役専任執行役員
石川 五十一郎	取締役専任執行役員
山口 五十二郎	取締役専任執行役員
北村 五十三郎	取締役専任執行役員
上野 五十四郎	取締役専任執行役員
小島 五十五郎	取締役専任執行役員
大塚 五十六郎	取締役専任執行役員
森田 五十七郎	取締役専任執行役員
山崎 五十八郎	取締役専任執行役員
斎藤 五十九郎	取締役専任執行役員
高木 六十郎	取締役専任執行役員
岡田 六十一郎	取締役専任執行役員
宮田 六十二郎	取締役専任執行役員
林 六十三郎	取締役専任執行役員
森 六十四郎	取締役専任執行役員
山本 六十五郎	取締役専任執行役員
佐々木 六十六郎	取締役専任執行役員
田中 六十七郎	取締役専任執行役員
鈴木 六十八郎	取締役専任執行役員
高橋 六十九郎	取締役専任執行役員
伊藤 七十郎	取締役専任執行役員
渡辺 七十一郎	取締役専任執行役員
松本 七十二郎	取締役専任執行役員
石川 七十三郎	取締役専任執行役員
山口 七十四郎	取締役専任執行役員
北村 七十五郎	取締役専任執行役員
上野 七十六郎	取締役専任執行役員
小島 七十七郎	取締役専任執行役員
大塚 七十八郎	取締役専任執行役員
森田 七十九郎	取締役専任執行役員
山崎 八十郎	取締役専任執行役員
斎藤 八十一郎	取締役専任執行役員
高木 八十二郎	取締役専任執行役員
岡田 八十三郎	取締役専任執行役員
宮田 八十四郎	取締役専任執行役員
林 八十五郎	取締役専任執行役員
森 八十六郎	取締役専任執行役員
山本 八十七郎	取締役専任執行役員
佐々木 八十八郎	取締役専任執行役員
田中 八十九郎	取締役専任執行役員
鈴木 九十郎	取締役専任執行役員
高橋 九十一郎	取締役専任執行役員
伊藤 九十二郎	取締役専任執行役員
渡辺 九十三郎	取締役専任執行役員
松本 九十四郎	取締役専任執行役員
石川 九十五郎	取締役専任執行役員
山口 九十六郎	取締役専任執行役員
北村 九十七郎	取締役専任執行役員
上野 九十八郎	取締役専任執行役員
小島 九十九郎	取締役専任執行役員
大塚 一百郎	取締役専任執行役員



평행이동 거리 h

현재 h: 2.74

현재 절단 기준: 움직인 도형 $P + (0, 0, h)$ 와 $z = 3$ 의 교집합은 원래 도형 P 의 $z = 3 - h$ 단면을 위로 올린 것입니다.

$$P_h \cap \{z = 3\} = (P \cap \{z = 3 - h\}) + (0, 0, h)$$

현재 절단면은 4각형입니다. 원래 도형 기준 $z=0.26$

$h < 2$ 또는 $h > 5$ 에서는 현재 움직이는 삼각기둥이 평면 $z = 3$ 과 만나지 않습니다. 하지만 전체 자취 V 는 $0 \leq h \leq 6$ 동안 지나간 모든 부분의 합 집합입니다.

단면 모양
오각형

단면 넓이
5

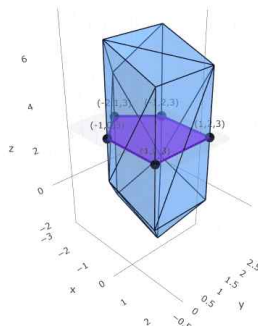
부피
35

2. 전체 자취 V 의 3차원 모습

초기 삼각기둥과 최종 삼각기둥 사이를 z 축 방향으로 쏘고 지나가면 아래의 볼록다면체가 됩니다. 평면 $z = 3$ 에서 잘린 단면은 보라색 오각형입니다.

전체 자취 V와 평면 $z=3$ 의 단면

전체 자취 V 와 평면 $z=3$ 의 단면



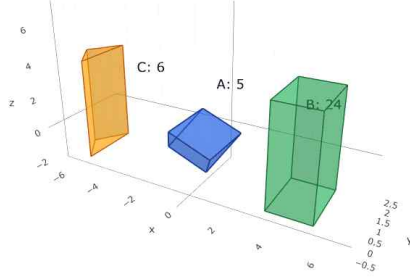
3. 부피 계산을 위한 등가 분해 3D

부피 계산은 카발리에리 원리로 생각하면 가장 직관적입니다. xy 평면으로의 정사영 넓이만큼 높이 6인 공통 이동량이 생기고, 여기에 원래 삼각기둥의 두께가 더해집니다.

$$\text{Vol}(V) = 6 \cdot \text{Area}(\text{proj}_{xy} P) + \text{Vol}(P)$$

부피 계산용 등가 분해: $5 + 24 + 6 = 35$

— A 경계 — B 경계 — C 경계

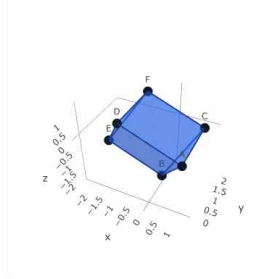


아래의 분해 그림은 "실제 위치를 그대로 자른 조각"이라기보다, 각 (x, y) 위치의 세로 길이를 보존하여 부피가 같게 만든 계산용 등가 조각입니다.

4. 각 조각별 3D 플롯과 부피

조각 A: 원래 삼각기둥 P

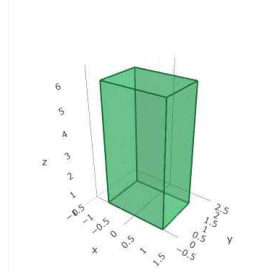
원래 삼각기둥 P



$$\frac{1}{2} |\det(B-A, C-A, D-A)| = 5$$

조각 B: 직사각형 정사영 2×2

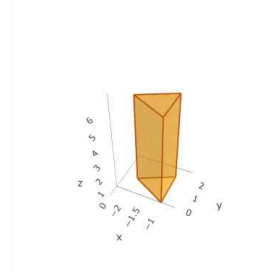
밑넓이 4, 높이 6



$$6 \times 4 = 24$$

조각 C: 왼쪽 삼각형 정사영

밑넓이 1, 높이 6



$$6 \times 1 = 6$$

5. 계산 과정 정리

점과 이동 벡터

$D = A + (-2, 0, 1)$, $E = B + (-2, 0, 1)$, $F = C + (-2, 0, 1)$ 이므로 원래 도형 P 는 삼각형 ABC 를 벡터 $q = (-2, 0, 1)$ 방향으로 민 삼각기둥입니다.

$$q = D - A = (-2, 0, 1)$$

원래 삼각기둥의 부피

$$\text{Vol}(P) = \frac{1}{2} |\det(B-A, C-A, q)| = \frac{1}{2} \left| \det \begin{pmatrix} -1 & 0 & -2 \\ 1 & 2 & 0 \\ -2 & 0 & 1 \end{pmatrix} \right| = \frac{1}{2} \cdot 10 = 5$$

평면 $z = 3$ 에서 생기는 최종 단면

$z = 3$ 을 지나간 모든 점의 (x, y) 자취는 원래 삼각기둥 P 의 xy 정사영과 같습니다. 따라서 단면 꼭짓점은 다음 오각형입니다.

순서	단면 꼭짓점
1	$(-2, 1, 3)$
2	$(-1, 0, 3)$
3	$(1, 0, 3)$
4	$(1, 2, 3)$
5	$(-1, 2, 3)$

$$\text{Area} = \underbrace{(2 \times 2)}_4 + \underbrace{\frac{1}{2} \cdot 2 \cdot 1}_1 = 5$$

전체 부피

$$\text{Vol}(V) = 6 \cdot 5 + 5 = 35$$

7. 최초 시뮬레이터에서 수정해야 할 사항을 서술하세요.

첫 번째로 나온 시뮬레이터의 가장 큰 문제는 입체도형을 나눈 모습이 원본에서 파생되는 것인지 직관적으로 와닿지 않는다는 것이다. 또한, 두 번째의 부피 계산을 자율적으로 나오도록 하였더니 교육과정을 벗어나 행렬의 행렬식 값으로 값을 구하였다.

8. 수정하기 위한 프롬프트를 (2번째 ~ n번째) 모두 작성하세요.

2번째: [문제 4-1]

먼저 삼각기둥 ABC-DEF를 이루는 면들에 대해 법선벡터와 넓이를 구하여 보자.

(i) 앞면 삼각형 ABC는 세 변의 길이가 각각 2, $\sqrt{6}$, $\sqrt{6}$ 인 이등변삼각형이므로 피타고라스의 정리를 이용하여 높이가 $\sqrt{5}$ 임을 알 수 있고, 따라서 그 넓이는 $\sqrt{5}$ 이다. 세 점 $(1, 0, 0)$, $(1, 2, 0)$, $(0, 1, -2)$ 가 이루는 평면의 방정식을 생각하면, 이에 수직인 벡터는 $(-2, 0, 1)$ 이다.

(ii) 마찬가지로, 사각형 ACFD의 넓이는 $2\sqrt{5}$ 이고, 세 꼭짓점을 골라 평면의 방정식을 생각하면, 이는 벡터 $(1, 0, 2)$ 와 수직임을 알 수 있다.

잘린 면의 모양은 오각형이며, 그 넓이는 (i), (ii)에서 구한 두 다각형을 평면 $z=3$ 에 정사영하여 얻을 수 있다. 먼저 삼각형 ABC와 $z=3$ 가 이루는 이면각을 θ_1 이라 두면,

$$\cos(\theta_1) = ((0, 0, 1) \cdot (-2, 0, 1)) / (|(0, 0, 1)| |(-2, 0, 1)|) = 1/\sqrt{5}.$$

마찬가지로 사각형 ACFD와 $z=3$ 가 이루는 각을 θ_2 라 두면,

$$\cos(\theta_2) = 1/\sqrt{5} \quad (\because \theta_1 + \theta_2 = \pi/2)$$

따라서 구하려는 잘린 면의 넓이는

$$\sqrt{5} \times 1/\sqrt{5} + 2\sqrt{5} \times 2/\sqrt{5} = 5$$

가 된다. (이 풀이는 연세대학교 선행학습영향평가보고서 보고서에 공개된 예시답안을 typingmonster를 통해 text화한 후 붙여넣은 것입니다.)

[문제 4-2]

그림에 있는 다면체 V 내부에서 위쪽에 보이는 삼각기둥 모양을 제외한 것을 V'이라 하자. 평면 $z=1$ 로 자른 아랫부분을 입체도형 끼워넣기로 생각하면 V'은 [문제 4-1]의 오각형을 단면으로 하고 높이가 6인 오각기둥과 같은 부피를 갖는다.

따라서 V의 부피는

$$6 \times (D \text{의 넓이}) + (\text{삼각기둥의 부피})$$

$$= 6 \times 5 + 1/2 \times 2\sqrt{5} \times \sqrt{5} = 35$$

이다.

이 풀이를 참고해서 풀이의 방향성을 수정해줘. 기존에 쪼갠 도형과 달리, 지금 풀이에 적혀있는 대로 쪼개주고, 그리고 각각이 원래 도형에서 나왔다는 것을 직관적으로 알기 쉽게 각 부분들이 합쳐져있는 그림도 같이 넣어줘. 4-1의 5각형 부분은 수정하지 않아도 좋아.

3번째 : 이 시뮬레이터에서 멘트들 중에서 수정한 티를 없애줬으면 좋겠어. 처음부터 그렇게 존재한 것처럼 멘트들을 수정해주고, 첨언은 html 내부가 아닌 외부 챗으로 해줘 예를 들어오각형은 그대로 유지 혹은 새로운 분리 이런 멘트들을 지우라는 거야.

9. 최종 시뮬레이터의 모습을 캡처해서 붙여넣으세요.

삼각기둥 ABC-DEF의 z축 평행이동 자취와 부피 분해

삼각기둥의 z축 평행이동으로 생기는 자취와 단면부피 계산을 함께 확인하는 시뮬레이터입니다.

0. 기본 설정

주어진 삼각기둥을 $P = ABC - DEF$ 라 두고, 이를 z축 방향으로 h만큼 옮긴 도형을 $P_h = P + (0, 0, h)$ 라 합니다.

$$A(1, 0, 0), B(0, 1, -2), C(1, 2, 0), D(-1, 0, 1), E(-2, 1, -1), F(-1, 2, 1)$$

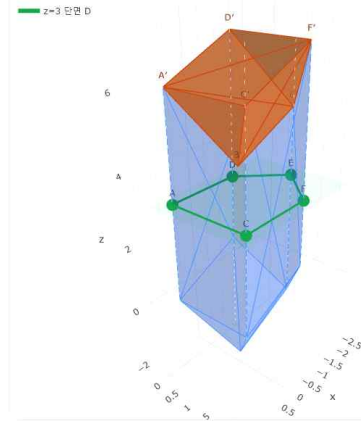
최종 다면체는 $h = 0$ 부터 $h = 6$ 까지 움직인 자취이므로

$$V = P + [0, 6]e_z$$

아래 그림에서는 파란색 부분은 "세로 방향으로 길이 h만큼 형성되는 부분", 주황색 부분은 "현재 위치의 삼각기둥 P_h "입니다.

1. h에 따라 생기는 자취

평행이동 거리 $h = 6.0$ 일 때의 자취



조작

평행이동 거리 h : 6.0

● V_h' : 길이 h 인 세로 섬유 부분 ● P_h : 현재 위치의 삼각기둥
● $z = 3$ 단면 오각형

오각형 단면 넓이

5

현재 세로 섬유 부분

30.0

현재 전체 부피

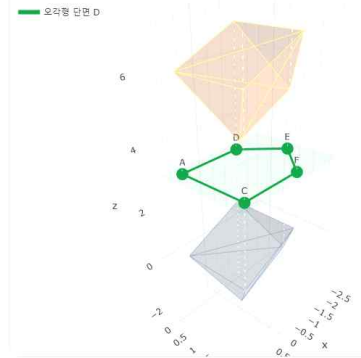
35.0

$$V_h = V_h' \cup P_h, \quad \text{Vol}(V_h) = 5h + 5$$

핵심은 h 가 변할 때 단면 오각형 자체가 움직이는 것이 아니라, 삼각기둥이 위로 이동하면서 각 (x, y) 위치에 세로 길이 h 의 자취가 쌓인다는 점입니다.

2. [문제 4-1] $z = 3$ 에서 잘린 단면

$z=3$ 단면: 오각형 D



단면 D의 꼭짓점과 넓이

최종 자취 V 를 평면 $z = 3$ 으로 자르면 다음 오각형이 생깁니다.

$$(-2, 1, 3), (-1, 0, 3), (1, 0, 3), (1, 2, 3), (-1, 2, 3)$$

풀이에서처럼 면의 정사영으로 보면, 삼각형 부분의 정사영 넓이가 1, 사각형 부분의 정사영 넓이가 4가 되어

$$S_D = 1 + 4 = 5$$

이 오각형을 D 라 두면, 부피 계산에서 V' 의 기준 단면이 됩니다.

3. 부피 분해: $V = V' \cup P_6$

최종 도형 V 는 다음 두 부분으로 나누어 보는 것이 가장 적관적입니다.

$$V' = \{(x, y, z) \mid (x, y) \in D, z_{\min}(x, y) \leq z \leq z_{\min}(x, y) + 6\}$$

각 (x, y) 에서 세로 길이가 항상 6인 부분입니다. 바닥이 기울어져 있지만, 세로 선유를 가지런히 하면 높이 6인 오각기둥과 같은 부피입니다.

$$P_6 = P + (0, 0, 6)$$

맨 위에 남은 부분입니다. 원래 삼각기둥 $ABC-DEF$ 가 그대로 위로 6만큼 이동한 것이므로 부피는 원래 삼각기둥의 부피와 같습니다.

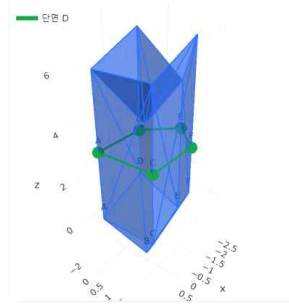
최종 도형 $V = V' \cup P_6$



4. 나누어진 각 도형의 3D 플롯

① V' : 세로 선유 길이 6인 부분

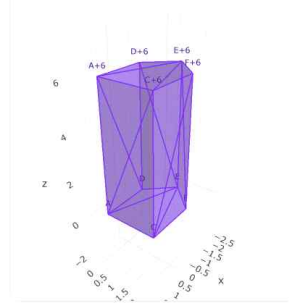
V' 만 보기



$$\text{Vol}(V') = 6 \times S_D = 6 \times 5 = 30$$

② 같은 부피의 높이 6 오각기둥

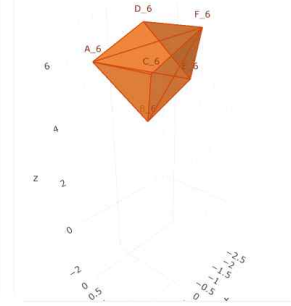
V' 와 같은 부피의 곧은 오각기둥



$$\text{Vol} = \text{밑넓이} \times \text{높이} = 5 \times 6 = 30$$

③ 위쪽 삼각기둥 P_6

위쪽 삼각기둥 P_6



$$\text{Vol}(P_6) = \frac{1}{2} \times 2\sqrt{5} \times \sqrt{5} = 5$$

5. 계산 과정 요약

단계	내용	값
4-1	평면 $z=3$ 으로 자른 단면 D	오각형
4-1	오각형 D 의 넓이	$S_D = 5$
4-2	아랫부분 V' 은 세로 길이 6의 선유를 오각형 D 전체에 세운 것과 같은 부피	$6 \times 5 = 30$
4-2	위쪽 삼각기둥은 원래 삼각기둥을 위로 6만큼 옮긴 것	5
결론	전체 부피	$30 + 5 = 35$

$$\text{Vol}(V) = 6 \times 5 + 5 = 35$$

10. 자신이 만든 시뮬레이터를 처음 본 사람이 사용할 수 있도록 사용 방법을 서술하세요.

1. 삼각기둥의 평행이동 및 자취 형성 관찰 (슬라이더 제어)

- 조작 방법: 화면 중앙의 '평행이동 거리 h ' 슬라이더를 마우스로 드래그하여 h 값을 0부터 6까지 수동으로 조절하며, 3D 그래프 위를 360도 드래그하여 생성되는 자취의 양상을 눈으로 확인한다.
- 기능: 삼각기둥이 z 축 방향으로 이동함에 따라 다면체 V 의 형성 과정을 직관적으로 관찰할 수 있음. 하단의 지표 수치를 통해 h 의 변화에 따른 '새로 생성된 부피'와 '전체 부피'의 값을 실시간으로 확인 가능함.

2. [문항 4-1] 분석: 특정 평면($z=3$)에서의 잘린 단면 확인

- 조작 방법: '2. $z=3$ 에서 잘린 단면' 영역의 3D 플롯을 마우스로 회전시켜 단면 관찰.
- 기능: 평행이동 궤적이 $z=3$ 평면과 교차하여 생성되는 오각형 단면(D)의 정확한 위치와 형태를 시각적으로 파악할 수 있음. 해당 단면이 부피 계산의 기준 밑면이 되는 원리와 면의 정사영을 통한 넓이 산출 과정을 이해하는데 도움을 줌.

3. [문항 4-2] 분석: 최종 다면체 부피 분해 시각화 및 인사이트 제공

- 조작 방법: '3. 부피 분해: $V = V' \cup P_h$ ' 에서 합쳐져있는 3D 플롯을 조작하여 삼각기둥과 지나온 부분의 결합 구조를 확인할 수 있다.
- 기능: 각각이 분해 되어있는 모습을 보여줌으로서 부피 계산의 전략을 제공함.

4. 분해된 도형의 개별 구조 및 등적 변형 비교 관찰

- 조작 방법: '4. 나누어진 각 도형의 3D 플롯' 영역에 나열 되어있는 3개의 소형 3D 플롯을 각각 마우스로 회전하며 1과 2를 비교하고, 3의 모양을 확인.
- 기능: 밑면이 기울어진 비스듬한 형태의 세로 자취 도형(V'')이 실제로는 높이가 6인 곧은 오각기둥과 부피가 완벽히 동일함을 그래픽으로 대조하여 보여줌. 이를 통해 정적분이나 카발리에리의 원리를 직관적으로 납득시키고, 최종 전체 부피 계산($30+5=35$)의 논리적 타당성을 직접 검증할 수 있음.

11. 시뮬레이터가 자신이 의도한대로 문제를 파악하거나 풀이과정에 대한 힌트를 얻거나 풀이에 대한 검토를 하는 등에 도움이 되었는지, 도움이 되거나 되지 않는 부분에 대해서 구체적으로 서술하세요.

이 문제를 포함한 직선 경로로 다면체가 움직인 자취의 부피를 구하는 문제에 모두 적용될 수 있는 인사이트를 제공할 수 있다고 생각한다. 정사영에 대한 아이디어와 부피의 분해가 정석적이기 때문이다.

[도움된 부분]

위에 글과 동일하다. 이 문제를 따라가는데 필요한 step들을 제공하여 시뮬레이션이 충분한 도움을 줄 수 있으리라 생각한다.

[도움되지 않은 부분]

단면에 대한 시뮬레이션이 순수하게 그 넓이를 계산하는데에 도움이 되지는 않기에 인사이트만을 제공한다는 점이 그 한계로 볼 수 있고, 사용자는 이 시뮬레이션을 보고 직접 부피를 계산하는 전략을 세우고 계산해야 한다.

[해결 방안]

사용자의 수학 능력 증진을 위해 이 한계점은 사용자가 넘어야하는 과제로서 남기는 게 더 좋을 듯 하다. (절대로 귀찮아서가 아니다::)

[마무리]

1. 두 문제에 대한 시뮬레이터를 기획하고 활용하여 문제를 해결하는 전 과정에서, 생성형 AI가 제공할 수 있는 가치와 반드시 인간(학생 본인)이 주도해야 하는 수학적 사고의 영역을 구분하여 서술하세요.

생성형 AI의 풀이는 빠르다는 것이 가장 큰 장점이다. 하지만 학생이 문제를 푸는 목적은 중요한 순간에 문제를 해결하기 위해 대비하는 것과 사고 과정을 학습하기 위함이다. 정작 AI가 힌트를 너무 많이 주게 되면 학생은 사고를 확장하려 하지 않는다. 그렇기 때문에 학생은 본인의 본 목적이 학습에 있음을 망각하지 말고 생성형 AI에게 문제를 맡기기 전에 본인이 문제의 해답을 탐색하는 시간을 갖고 그 다음에 AI를 돌려야 한다. 그 이유는 정작 문제를 푸는 수학적 사고의 주체는 학생이 되어야 하기 때문이며, AI는 단순히 참고용으로 사용해야 한다.

기술적 혹은 시각화를 요하는 작업은 생성형 AI에 맡기되, 그 결과를 해석하고 논하는 것은 사람이 해야 한다.

2. 복잡한 수학 문제를 AI와 협업하여 시뮬레이터로 구현해 본 경험이, 향후 자신이 희망하는 진로 분야에서 실무적인 문제를 해결하는 데 어떤 시사점을 주는지 서술하세요.

이 추세는 어느 정도 물리학과와 맞닿아있는 듯 하다. 요즘 물리학 혹은 공학 대학원에서는 이론을 배운 이후 문제가 안 풀릴 경우 모두 수치해석으로 문제를 해결하고 있는 추세이다. 모두가 코딩하고, 컴퓨터에게 문제 풀이를 시키는 시대이다. 이와 같은 생성형 AI는 사람에게 새로운 인사이트를 제공하기도 하고, 사람이 풀기에 번거로운 계산도 해낸다. 따라서, 실무적인 문제들을 해결하는 사람들이 생성형 AI로 대체될 것이라는 전망을 주는 듯 하다.

3. 보고서를 작성하며 느낀점을 간단히 작성하세요.

HTML에서 3D가 제대로 구현됐을 때 기분이 굉장히 좋았다. 예상한 것보다 퀄리티가 굉장히 높았고, 제시한 명령을 굉장히 잘 따라서 기획한 것 보다 좋은 결과물을 뽑아내었다. 그러나 한 편으로는 이렇게 AI가 발전해서 풀이를 잘 해준다면 정녕 우리 아래 세대들은 학원을 다니지 않고 그 학원비로 GPT pro를 돌리지 않을까 싶었다. 이와 같은 생성형 AI가 사교육 의존도를 낮출 수 있을 것 같다는 생각이 들었다.